Communications in Distributed Systems

Shubhangi Shreya
Assistant Professor(I)
School of Computer Engineering
KIIT Deemed to be University
Bhubaneswar

UNIT-3: Synchronization and Processes

- Clock Synchronization
- Clock Synchronization Algorithms
- Mutual Exclusion Algorithms
- Election Algorithms
- Atomic Transactions & Modeling
- Atomic Transaction Implementation
- Concurrency Control Algorithms in Atomic Transaction

Some critical challenges and principles of synchronization in distributed systems

1.Information Scattered Across Multiple Machines:

- Decentralization: In distributed systems, data and control are distributed across many machines. Algorithms must be designed to work without gathering all the data at a single point, avoiding bottlenecks and single points of failure.
- Local Decision Making: Processes rely on information that is available locally and communicate with other processes to coordinate. This can involve passing messages or synchronizing state without central oversight.

2.Decisions Based on Local Information:

- Autonomy: Each process or node operates based on its local view of the system. This local information may include the state of the process itself and recent communications with other processes.
- Consistency Models: Distributed systems often use consistency models (like eventual consistency) to ensure that, despite local decision-making, the system as a whole remains consistent over time.

3.Avoiding Single Points of Failure:

- Fault Tolerance: Distributed systems are designed to tolerate failures by ensuring that the failure of one machine does not bring down the entire system. This is achieved through redundancy, replication, and robust failure recovery mechanisms.
- Failover Mechanisms: Techniques such as leader election and backup nodes ensure that if one node fails, another can take over its responsibilities.

4.Lack of Global Clock or Precise Time Source:

- Logical Clocks: Since global clocks are impractical, distributed systems use logical clocks (like Lamport timestamps) to order events and ensure that the system can track causality and consistency without synchronized time.
- Vector Clocks: For a finer granularity of event ordering and to track causality more precisely, vector clocks can be used. They provide a way to capture the partial ordering of events in a distributed system.

Centralized Resource Allocation: Problems

1.**Heavy Burden on a Single Process:**

- Scenario: In a centralized approach, all resource requests (e.g., for I/O devices) are sent to a single manager process. This manager is responsible for examining these requests, making decisions, and allocating resources.
- Problem: In a large distributed system, this manager would be overwhelmed by the sheer volume of requests and information it needs to handle. This centralized process becomes a bottleneck because it must manage all resource requests from potentially thousands of other processes. As the system scales up, the load on this single process becomes impractical and inefficient.

2.Single Point of Failure:

- Scenario: If the centralized manager process fails, it can result in significant disruptions. All resource allocation decisions depend on this one manager, so its failure can bring the whole system to a halt. For example, if the resource allocator crashes, none of the processes can get the resources they need, causing a cascade of failures or blocking.
- Problem: This makes the system unreliable because the failure of one component (the manager) affects the entire distributed system. Ideally, a distributed system should continue to function even if some of its components fail. If a single point of failure can cause widespread disruption, it contradicts the goal of high reliability and fault tolerance.

Challenges in a Distributed System:

- **Inconsistent Timestamps**: In a distributed environment where multiple machines may be involved, each machine could have its own local clock. If the clocks are not synchronized perfectly, the timestamps on files could be inconsistent or misleading. For example, one machine might think a file was modified at a certain time, while another might see a different timestamp due to clock discrepancies.

- **Inconsistent Build Results**: If make were running across a distributed system, inconsistent timestamps could lead to incorrect build decisions, such as not recompiling a file that should be updated or unnecessarily recompiling files that haven't changed.

Logical Clocks

Introduction to Computer Timers

- Most computers come equipped with a circuit referred to as a clock, although a more accurate term would be a "timer." A computer timer typically consists of a precisely machined quartz crystal that oscillates at a specific frequency based on its characteristics. This frequency determines how the timer functions.

Structure of a Computer Timer

- A computer timer usually has two registers: a counter and a holding register. Each oscillation of the quartz crystal decreases the counter by one. When the counter reaches zero, an interrupt is generated, and the counter is reloaded from the holding register. This setup allows the timer to generate interrupts at any desired frequency, such as 60 times per second. Each interrupt is called a clock tick.

Initialization and Updating of the Software Clock

- Upon system boot, the operator typically enters the current date and time, which is then converted to the number of ticks since a known starting date and stored in memory. The system clock is updated at every clock tick by incrementing this stored time.

Single vs. Multiple Clocks

- For a single computer with a single clock, slight inaccuracies are generally inconsequential as all processes use the same clock, maintaining internal consistency. Problems arise with multiple CPUs, each having its own clock. Even though crystal oscillators are stable, variations in their frequencies cause clocks to drift apart, leading to clock skew. This discrepancy can cause issues in programs relying on correct and consistent timestamps across different machines.

Logical Clocks vs. Physical Clocks

- For many purposes, internal consistency of clocks suffices, even if the time does not match real-world time exactly. Clocks in this context are called **logical clocks**.
- When clocks must align closely with real-world time, they are referred to as **physical clocks**.

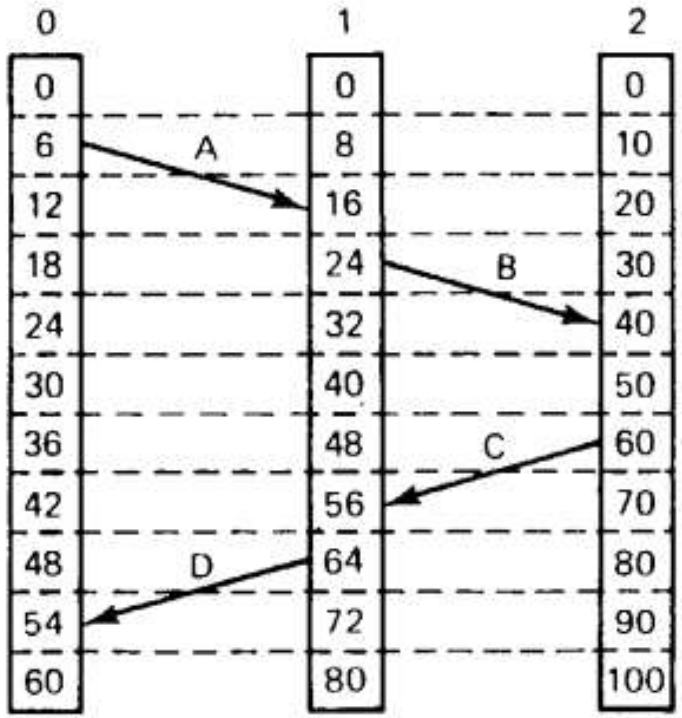
Lamport's Happens-Before Relation

To synchronize logical clocks, Lamport introduced the happens-before relation, denoted as "a happens before b" (a → b). This relation is observed in two scenarios:

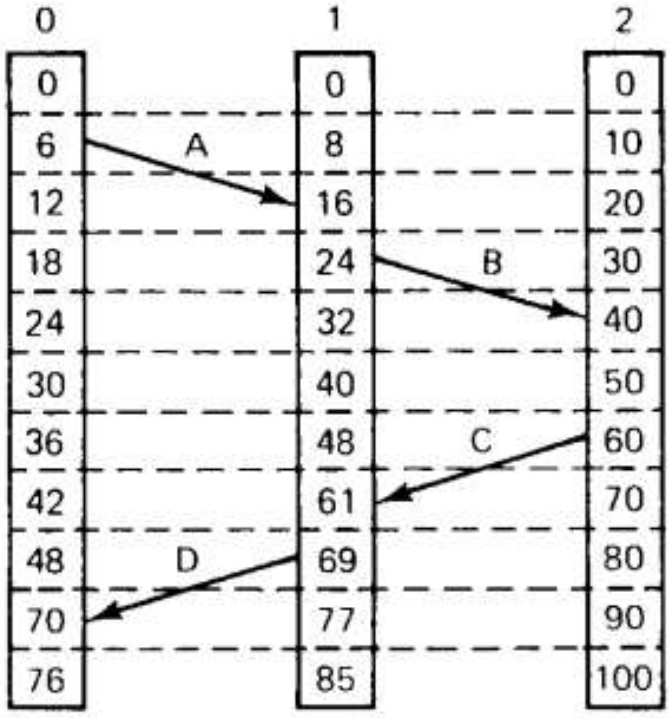
- If events a and b occur in the same process and a happens before b, then a → b.
- If event a is the sending of a message by one process and event b is its receipt by another process, then a → b since a message cannot be received before it is sent.

The happens-before relation is transitive, meaning if a → b and b → c, then a → c. If events x and y occur in different processes without exchanging messages, they are concurrent, meaning neither x → y nor y → x holds.

Lamport solution



a)clock run at different rate



b)Lamport algo correct the clocks

Way to assign times to events in a distributed system

- If a happens before b in the same process, then C(a) < C(b).
- If a and b are the sending and receiving of a message, then C(a) < C(b).
- All events a and b must have unique times, C(a) ≠ C(b).

Physical Clocks

Importance of Physical Clocks

While Lamport's algorithm provides an unambiguous ordering of events, the assigned time values do not necessarily reflect the actual occurrence times. In systems where real-time accuracy is critical, such as in real-time systems, external physical clocks are essential. Having multiple physical clocks enhances efficiency and redundancy, but it introduces two significant challenges:

- Synchronizing these clocks with real-world time.
- Synchronizing the clocks with each other.

Measurement of Time

Accurate time measurement is complex. Historically, time measurement was astronomical, based on the solar day—the interval between two consecutive transits of the sun, which reaches its highest point in the sky at noon each day. A solar second was defined as exactly 1/86400th of a solar day.

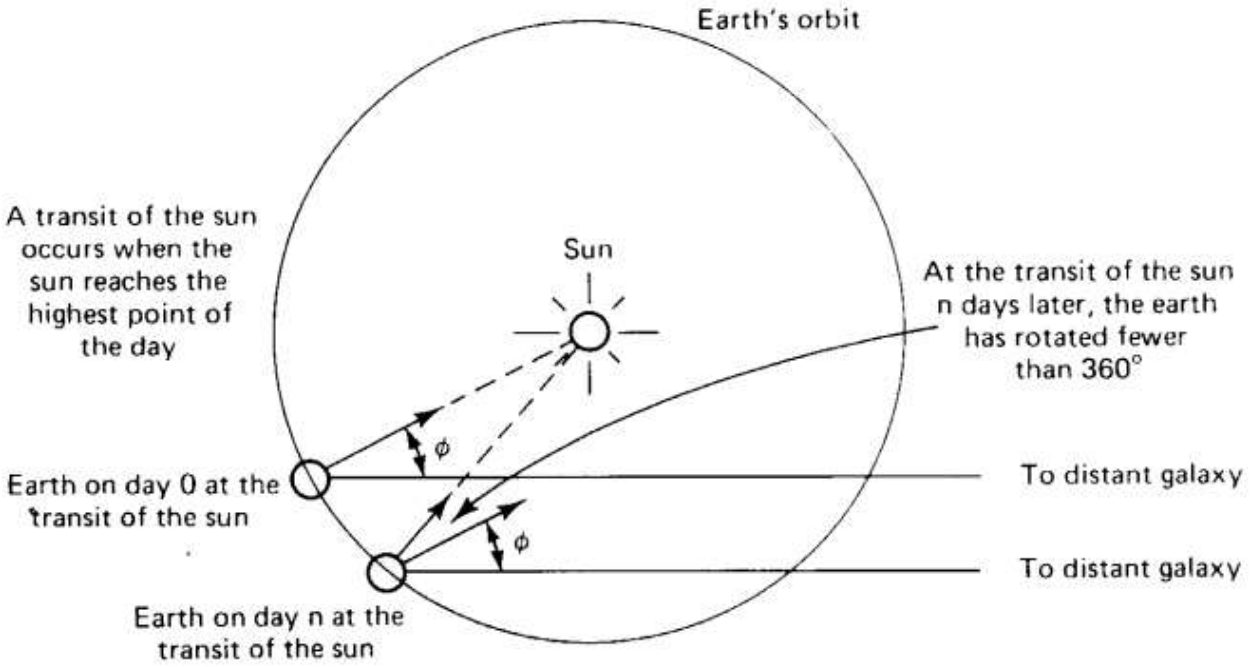
Variations in Earth's Rotation

In the 1940s, it was discovered that the Earth's rotation period is not constant due to tidal friction and atmospheric drag. Geological studies suggest that 300 million years ago, there were about 400 days in a year, indicating that the Earth's rotation has slowed down. Additionally, short-term variations in day length occur due to turbulence in the Earth's core.

Mean Solar Second and Atomic Clocks

Astronomers calculated the mean solar second by averaging the length of many days. With the invention of the atomic clock in 1948, timekeeping became more precise. The atomic second was defined based on the cesium 133 atom's transitions, making the atomic second equal to the mean solar second at the time of its introduction.

Computation of the mean solar day



International Atomic Time (TAI)

- Approximately 50 laboratories worldwide use cesium 133 clocks to measure time. These laboratories report their clock ticks to the Bureau International de l'Heure (BIH) in Paris, which averages them to produce International Atomic Time (TAI). TAI is the mean number of cesium 133 clock ticks since January 1, 1958, divided by 9,192,631,770.

Leap Seconds and Universal Coordinated Time (UTC)

- Due to the Earth's slowing rotation, 86,400 TAI seconds are now about 3 milliseconds shorter than a mean solar day. To prevent discrepancies, BIH introduces leap seconds when the difference between TAI and solar time reaches 800 milliseconds. This adjustment maintains synchronization with the sun's apparent motion, resulting in Universal Coordinated Time (UTC), the modern civil timekeeping standard.

Distribution and Accuracy of UTC

- UTC is disseminated through various means, including shortwave radio stations like WWV in Fort Collins, Colorado, and MSF in Rugby, Warwickshire. These broadcasts offer UTC with an accuracy of about ±1 millisecond, though atmospheric conditions can reduce practical accuracy to ±10 milliseconds. Satellites also provide UTC services with high accuracy, requiring precise knowledge of sender-receiver positions to account for signal propagation delays.

Q&A

Homework Questions

1) What is synchronization in distributed system?
2) What is clock synchronization?
3) Logical clock vs Physical clock.
4) Name at least three sources of delay that can be introduced between WWV broadcasting the time and the processors in a distributed system setting their internal clocks.
5) Consider the behavior of two machines in a distributed system. Both have clocks that are supposed to tick 1000 times per millisecond. One of them actually does, but the other ticks only 990 times per millisecond. If UTC updates come in once a minute, what is the maximum clock skew that will occur?

Thank You!